



C avancé

TP3 - Fonctions variadiques et pointeurs de fonctions

Maxime MARIA

Respectez bien les consignes et testez le bon fonctionnement de tous vos programmes ! Pensez aussi à sauvegarder régulièrement ainsi qu'à garder tous vos codes !

1 Fonctions variadiques

1. Écrivez une fonction variadique, `printAdr`, qui affiche des adresses passées en paramètres (une par ligne).
2. Écrivez une fonction variadique, `initInts`, qui initialise des entiers. Cette fonction prend donc en paramètres des couples A/B où B est la valeur à assigner à A .
3. Vous allez maintenant coder votre propre `printf` (très simplifié) : `myPrintf`. Vous supposerez que la chaîne à afficher ne dépasse pas 2048 caractères. Pour ne pas utiliser `printf` pour afficher cette chaîne (ce serait dommage...), vous utiliserez la fonction `fwrite`, sur le flux `stdout`. Vous ajouterez la gestion des types suivants une par une et vérifierez le bon fonctionnement de votre fonction au fur et à mesure :
 - les caractères ("%c");
 - les chaînes de caractères ("%s");
 - les entiers ("%d", "%o" et "%x") : pour cela, vous devrez utiliser la fonction `sprintf` (oui, utiliser une fonction variadique dans une autre c'est un peu bizarre mais bon...);
 - les flottants ("%f") : une nouvelle fois, vous devrez utiliser la fonction `sprintf`.

Dans le cas où le format n'est pas valide, vous afficherez simplement le paramètre suivant %.

4. (*Bonus*) Codez maintenant votre `scanf` ! Cette fois, il faudra utiliser la fonction `fread` sur le flux `stdin`.

2 Pointeurs de fonctions

Pointeurs sur des fonctions `math.h`

Vous devez écrire un programme qui, à partir d'un nombre réel passé en argument, calcule le résultat de l'application des fonctions de `math.h` suivantes : `sqrtf`, `expf`, `exp2f`, `logf`, `log2f`, `cosf`, `sinf` et `tanf`. Par exemple, pour 7.9, l'affichage serait :

```
sqrt(7.900000) = 2.810694
expf(7.900000) = 2697.282471
exp2(7.900000) = 238.856461
log(7.900000) = 2.066863
log2(7.900000) = 2.981853
cos(7.900000) = -0.046002
sinf(7.900000) = 0.998941
tanf(7.900000) = -21.715067
```

Pour cela, vous devrez utiliser un tableur de pointeurs de fonctions (contenant les adresses des fonctions sus-nommées) ainsi qu'une boucle.

N.B. : Vous devez compiler avec l'option `-lm` pour lier la bibliothèque « math » et pouvoir utiliser les fonctions de `math.h`.

Approximation d'intégrales

La méthode des trapèzes¹ est une technique permettant l'approximation d'une intégrale. Celle-ci consiste à considérer la région sous le graphe d'une fonction $f(x)$ comme un trapézoïde afin de calculer son aire :

$$\int_a^b f(x) dx \approx (b-a) \frac{f(a) + f(b)}{2}$$

Ainsi, en discrétisant de manière uniforme l'intervalle $[a, b]$ en N sous-intervalles, il est possible de calculer une approximation de l'intervalle telle que :

$$\begin{aligned} \int_a^b f(x) dx &\approx \frac{\Delta_x}{2} \sum_{i=1}^N (f(x_{i-1}) + f(x_i)) \\ &\approx \frac{\Delta_x}{2} (f(x_0) + 2f(x_1) + 2f(x_2) \cdots + 2f(x_{N-1}) + f(x_N)) \\ &\approx \Delta_x \left(\sum_{i=1}^{N-1} f(x_i) + \frac{f(x_0) + f(x_N)}{2} \right) \end{aligned}$$

où $\Delta_x = \frac{b-a}{N}$.

Le fichier `TP4_ptr_fct_integral.c` disponible sur la plateforme pédagogique contient l'algorithme permettant de réaliser cette approximation pour $f(x) = \sqrt{x}$.

1. Modifiez le programme de sorte à ce que la fonction `approxIntegral` utilise un pointeur de fonction et qu'elle puisse fonctionner avec n'importe quelle fonction $f(x)$.
2. Vérifiez le bon fonctionnement de votre programme en comparant le résultat de votre fonction avec celui donné ici : <https://fr.planetcalc.com/5494/>.

(Il y a trop de fois le mot « fonction » dans cet exercice... ^^)

1. https://fr.wikipedia.org/wiki/M%C3%A9thode_des_trap%C3%A8zes